1. Template General

```cpp
#include <bits/stdc++.h>
using namespace std;
#define fast ios::sync_with_stdio(false); cin.tie(nullptr); cout.tie(nullptr)
#define endl "\n"
#define pb push_back
#define all(x) (x).begin(), (x).end()
#define sz(x) (int)(x).size()

// Macros para operaciones bitwise
#define isOn(S, j) (S & (1 << j))
#define setBit(S, j) (S |= (1 << j))
#define clearBit(S, j) (S &= ~(1 << j))
#define toggleBit(S, j) (S ^= (1 << j))
#define lowBit(S) (S & (-S))
#define setAll(S, n) (S = (1 << n)-1)

#define modulo(S, N) ((S) & (N-1)) // Retorna S % N, donde N es una potencia de 2
#define isPowerOfTwo(S) (!(S) & (S-1))
#define nearestPowerOfTwo(S) (1 << lround(log2(S)))
#define turnOffLastBit(S) ((S) & (S-1))
#define turnOnLastZero(S) ((S) | (S+1))
#define turnOffLastConsecutiveBits(S) ((S) & (S+1))
#define turnOnLastConsecutiveZeroes(S) ((S) | (S-1))


typedef long long ll;
typedef short int si;
typedef unsigned long long ull;
typedef long double ld;
typedef unsigned int ui;
typedef string str;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;


const int INFI = INT_MAX;
const long INFL = LONG_MAX;
const long long INFLL = LLONG_MAX;


```

```
int main(){
    fast;

    si n; cin >> n;
    vector<int> v;
    v.resize(n);
    for (int i = 0; i < n; ++i) cin >> v[i];

    cout << fixed << setprecision(10);

    return 0;
}
```

2. Bitmask y Binarios

```
  ll binpow(ll a, ll b){
    ll res = 1;
    ll counter = 0;
    while (b > 0){
      bitset<16> binary(b);
      if ( b& 1 ) res = res * a;
      a + a * a;
      counter++;
      b >>=1;
    }
    return res;
  }

  ll binpowmod(ll a, ll b, ll m){
    a \%= m;
    ll res = 1;
    while ( b > 0 ){
      if ( b & 1 ) res = res * a \% m;
      a = a * a \% m;
      b >>= 1;
    }
    return res;
  }


  bitset<8> b1; // 00000000
  bitset<8> b2(5); // 00000101
  bitset<8> b3(string("1101")); // 00001101
```

```
  bitset<8> b("10110010")
  b[0] // 0 (least significant bit)
  b[7] // 1 (most significant bit)
  b[0] = 1; // Set bit 0 10110011
  b.set(3); // set bit at index 3 10111011
  b.reset(1); // reset bit at index 1
  b.flip(2); // toggle bit at index 2 10111101
  b.flip();

  b.any() // True if any bit == 1
  b.none() // true if all bits == 0
  b.all() // true if all bits == 1
  b.count() // number of bits == 1
  b.size() // number of bits in bitset

  b.to_ulon() // unsigned long
  b.to_ullong() // unsigned long long
  b.to_string() // string

// Obtener los subconjuntos de una mascara en binario
vector<int> subset_bitmask(int mask){
    vector<int> subsets;
    for (int subset = mask; subset; subset = (mask & (subset
       -1))) subsets.push_back(subset);
    return subsets;
}
```

3. Swap dos números

```
void swap_two_iters(vector<int> vec, int n){
    const auto itr = find(all(vec), n);
    auto indx = distance(vec.begin(), itr);
    iter_swap(vec.begin() + n, itr);
    iter_swap(vec.begin() + n, vec.begin() + indx);
}

int num = 2;
const auto itr = find(all(v), num);
auto indx = distance(v.begin(), itr);
iter_swap(v.begin() + num, v.begin() + indx);
```